

Sistemas Inteligentes– I

Lección. Agrupamiento Algoritmo K-MEANS

Luis Fernando Castillo Ossa

Índice

- Algoritmo K MEANS
- Ejemplos
- Implementación en TensorFlow

INTRODUCTION-

What is clustering?

- **Clustering** is the classification of objects into different groups, or more precisely, the partitioning of a data set into subsets (clusters), so that the data in each subset (ideally) share some common trait - often according to some defined distance measure.

Types of clustering:

1. **Hierarchical algorithms**: these find successive clusters using previously established clusters.

1. **Agglomerative ("bottom-up")**: Agglomerative algorithms begin with each element as a separate cluster and merge them into successively larger clusters.

2. **Divisive ("top-down")**: Divisive algorithms begin with the whole set and proceed to divide it into successively smaller clusters.

2. **Partitional clustering**: Partitional algorithms determine all clusters at once. They include:

- **K-means and derivatives**

- Fuzzy c-means clustering

- QT clustering algorithm

Common Distance measures:

■ *Distance measure* will determine how the *similarity* of two elements is calculated and it will influence the shape of the clusters.

They include:

1. The Euclidean distance (also called 2-norm distance) is given by:

$$d(x, y) = \sum_{i=1}^p |x_i - y_i|$$

2. The Manhattan distance (also called taxicab norm or 1-norm) is given by:

$$d(x, y) = \sqrt{\sum_{i=1}^p |x_i - y_i|^2}$$

3. The maximum norm is given by:

$$d(x, y) = \max_{1 \leq i \leq p} |x_i - y_i|$$

4. The Mahalanobis distance corrects data for different scales and correlations in the variables.
5. Inner product space: The angle between two vectors can be used as a distance measure when clustering high dimensional data
6. Hamming distance (sometimes edit distance) measures the minimum number of substitutions required to change one member into another.

K-MEANS CLUSTERING

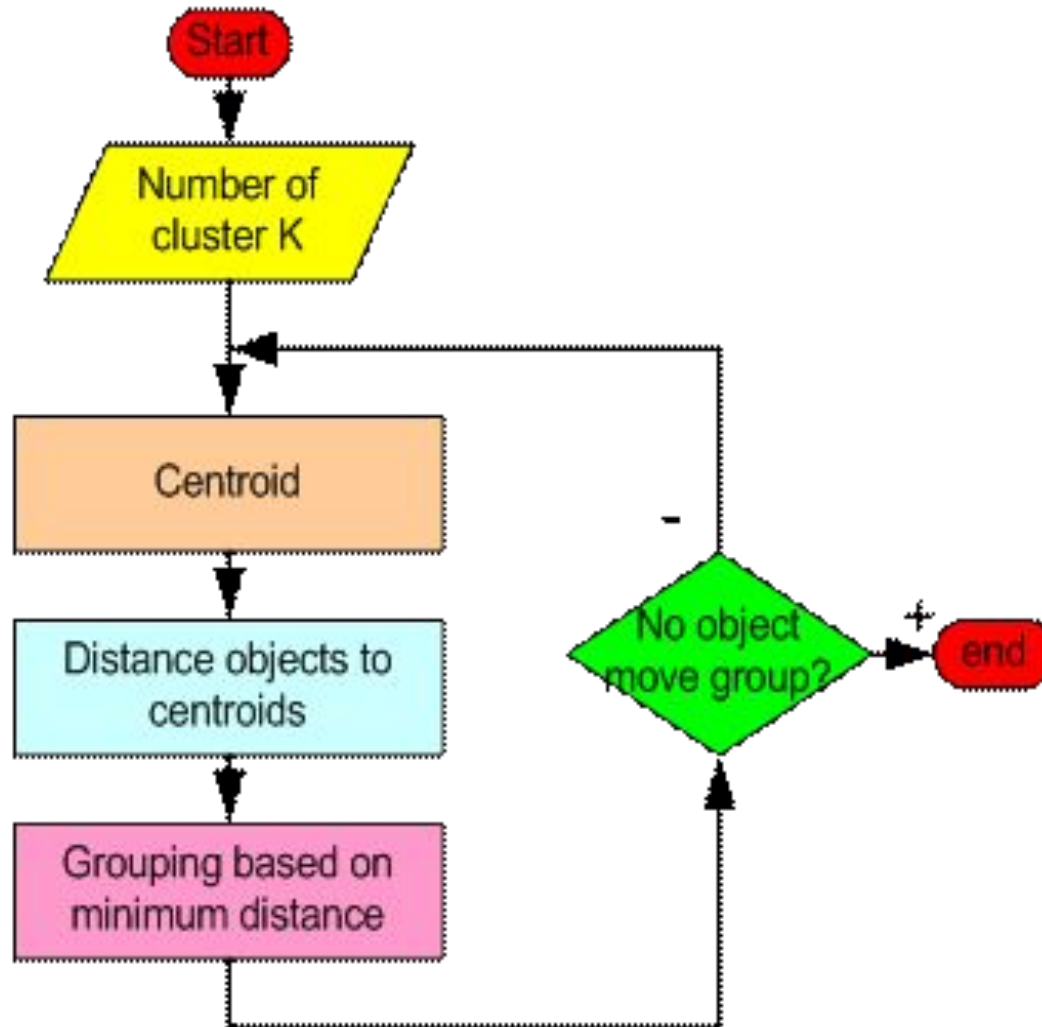
- The **k-means algorithm** is an algorithm to cluster n objects based on attributes into k partitions, where $k < n$.
- It is similar to the expectation-maximization algorithm for mixtures of Gaussians in that they both attempt to find the centers of natural clusters in the data.
- It assumes that the object attributes form a vector space.

- An algorithm for partitioning (or clustering) N data points into K disjoint subsets S_j containing data points so as to minimize the sum-of-squares criterion

$$J = \sum_{j=1}^K \sum_{n \in S_j} |x_n - \mu_j|^2,$$

where x_n is a vector representing the the n^{th} data point and μ_j is the geometric centroid of the data points in S_j .

How the K-Mean Clustering algorithm works?



Algorithm

■ **Step 1:** Begin with a decision on the value of k = number of clusters .

■ **Step 2:**

Put any initial partition that classifies the data into k clusters. You may assign the training samples randomly, or systematically as the following:

1. Take the first k training sample as single-element clusters

2. Assign each of the remaining $(N-k)$ training sample to the cluster with the nearest centroid. After each assignment, recompute the centroid of the gaining cluster.

- **Step 3:** Take each sample in sequence and compute its distance from the centroid of each of the clusters. If a sample is not currently in the cluster with the closest centroid, switch this sample to that cluster and update the centroid of the cluster gaining the new sample and the cluster losing the sample.
- **Step 4 .** Repeat step 3 until convergence is achieved, that is until a pass through the training sample causes no new assignments.

A Simple example showing the implementation of k-means algorithm (using $K=2$)

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5



Step 1:

Initialization: Randomly we choose following two centroids ($k=2$) for two clusters.

In this case the 2 centroid are: $m_1=(1.0,1.0)$ and $m_2=(5.0,7.0)$.

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

	Individual	Mean Vector
Group 1	1	(1.0, 1.0)
Group 2	4	(5.0, 7.0)

Step 2:

- Thus, we obtain two clusters containing:
 $\{1,2,3\}$ and $\{4,5,6,7\}$.

- Their new centroids are:

$$m_1 = \left(\frac{1}{3}(1.0 + 1.5 + 3.0), \frac{1}{3}(1.0 + 2.0 + 4.0) \right) = (1.83, 2.33)$$

$$m_2 = \left(\frac{1}{4}(5.0 + 3.5 + 4.5 + 3.5), \frac{1}{4}(7.0 + 5.0 + 5.0 + 4.5) \right)$$

$$= (4.12, 5.38)$$

Individual	Variable 1	Variable 2
1	1.0	1.0
2	1.5	2.0
3	3.0	4.0
4	5.0	7.0
5	3.5	5.0
6	4.5	5.0
7	3.5	4.5

Individual	Centroid 1	Centroid 2
1	0	7.21
2 (1.5, 2.0)	1.12	6.10
3	3.61	3.61
4	7.21	0
5	4.72	2.5
6	5.31	2.06
7	4.30	2.92

$$d(m_1, 2) = \sqrt{|1.0 - 1.5|^2 + |1.0 - 2.0|^2} = 1.12$$

$$d(m_2, 2) = \sqrt{|5.0 - 1.5|^2 + |7.0 - 2.0|^2} = 6.10$$

Step 3:

■ Now using these centroids we compute the Euclidean distance of each object, as shown in table.

■ Therefore, the new clusters are:

$\{1,2\}$ and $\{3,4,5,6,7\}$

■ Next centroids are:
 $m1=(1.25,1.5)$ and $m2 = (3.9,5.1)$

$= (4.12, 5.38)$

Individual	Centroid 1	Centroid 2
1	1.57	5.38
2	0.47	4.28
3	2.04	1.78
4	5.64	1.84
5	3.15	0.73
6	3.78	0.54
7	2.74	1.08

■ Step 4:

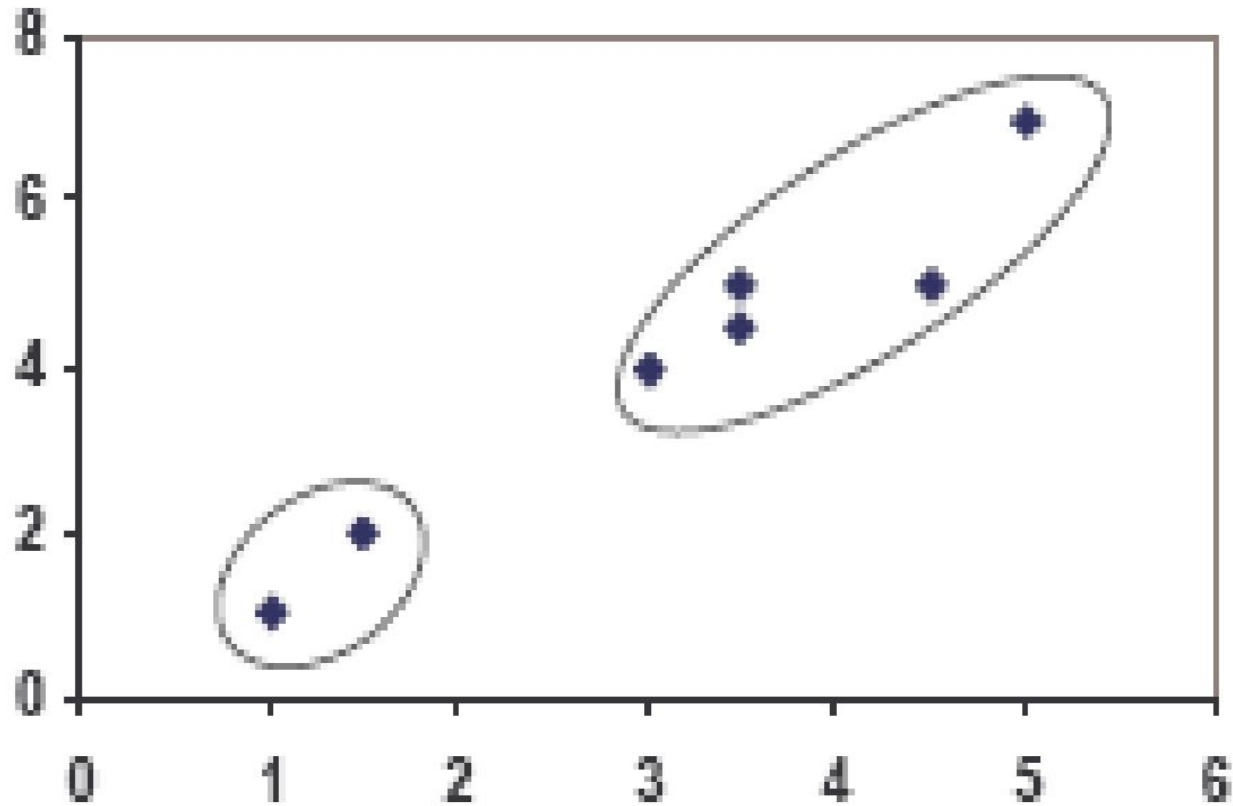
The clusters obtained are:

$\{1,2\}$ and $\{3,4,5,6,7\}$

- Therefore, there is no change in the cluster.
- Thus, the algorithm comes to a halt here and final result consist of 2 clusters $\{1,2\}$ and $\{3,4,5,6,7\}$.

Individual	Centroid 1	Centroid 2
1	0.58	5.02
2	0.58	3.92
3	3.05	1.42
4	6.68	2.20
5	4.18	0.41
6	4.78	0.61
7	3.75	0.72

PLOT



(with $K=3$)

Individual	$m_1 = 1$	$m_2 = 2$	$m_3 = 3$	cluster
1	0	1.11	3.81	1
2	1.12	0	2.5	2
3	3.81	2.5	0	3
4	7.21	8.10	3.81	3
5	4.72	3.81	1.12	3
6	5.31	4.24	1.80	3
7	4.30	3.20	0.71	3

C_3

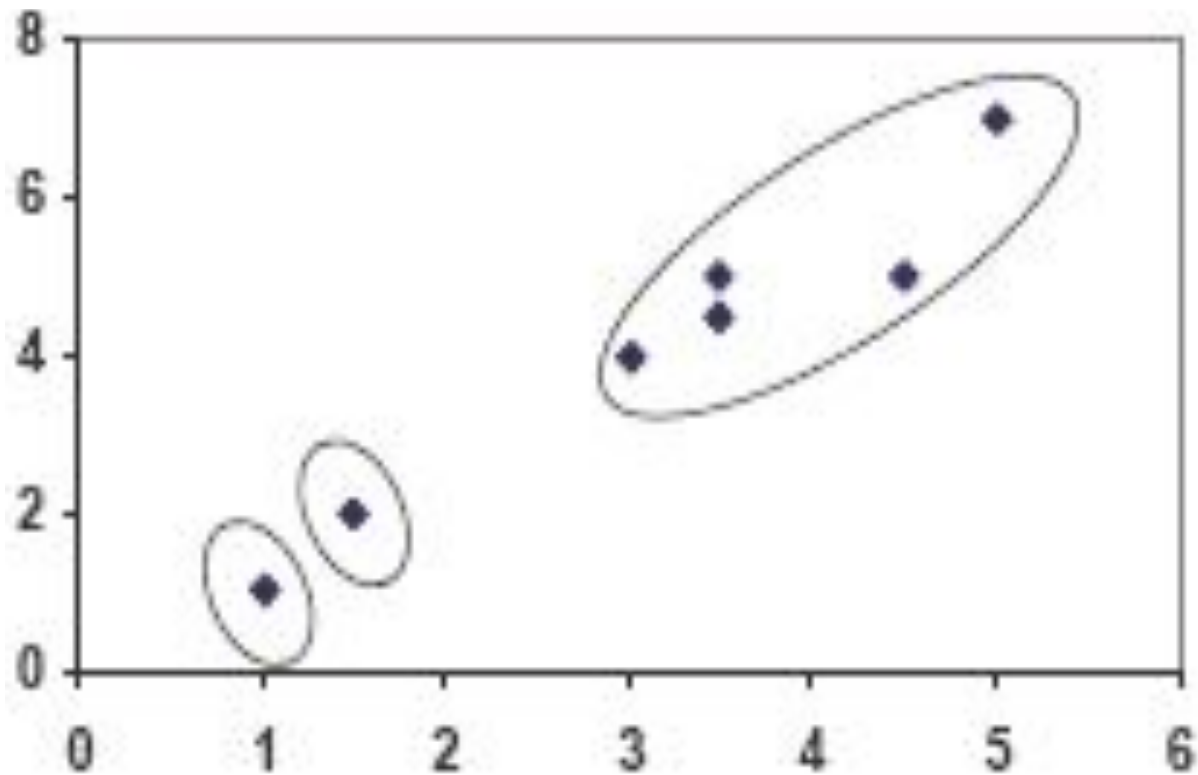
clustering with initial centroids (1, 2, 3)

Step 1

Individual	m_1 (1.0, 1.0)	m_2 (1.5, 2.0)	m_3 (3.9, 5.1)	cluster
1	0	1.11	5.02	1
2	1.12	0	3.92	2
3	3.81	2.5	1.42	3
4	7.21	8.10	2.20	3
5	4.72	3.81	0.41	3
6	5.31	4.24	0.81	3
7	4.30	3.20	0.72	3

Step 2

PLOT



K-Means Clustering Tensor Flow

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import os

#Evitar mensajes de aviso
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'

num_puntos = 800
num_clusters = 3
num_iteraciones = 200

#Generar los datos y seleccionar los centroides de manera aleatoria
puntos = tf.constant(np.random.uniform(0, 10, (num_puntos, 2)))
centroides = tf.Variable(tf.slice(tf.random_shuffle(puntos), [0, 0], [num_clusters, -1]))

#Aumentar una dimensión para poder restar
puntos_expand = tf.expand_dims(puntos, 0)
centroides_expand = tf.expand_dims(centroides, 1)

# Calcular la distancia euclídea y obtener la asignación de cada punto con el número de cluster más cercano
distancias = tf.sqrt(tf.reduce_sum(tf.square(tf.subtract(puntos_expand, centroides_expand)), 2))
vector_dist_minimas = tf.argmin(distancias, 0)

#Se calculan los nuevos centroides
lista = tf.dynamic_partition(puntos, tf.cast(vector_dist_minimas, tf.int32), num_clusters)
nuevos_centroides = [tf.reduce_mean(punto, 0) for punto in lista]

#Se asignan los centroides calculados a la variable centroides
centroides_actualizados = tf.assign(centroides, nuevos_centroides)

#Se inicia una sesión y se ejecuta el algoritmo
init = tf.global_variables_initializer()

with tf.Session() as sesion:
    sesion.run(init)
    for i in range(num_iteraciones):
        _, valores_centroides, valores_puntos, valores_distancias = sesion.run([centroides_actualizados, centroides, puntos, vector_dist_minimas])
        print("Centroides finales:\n",valores_centroides)

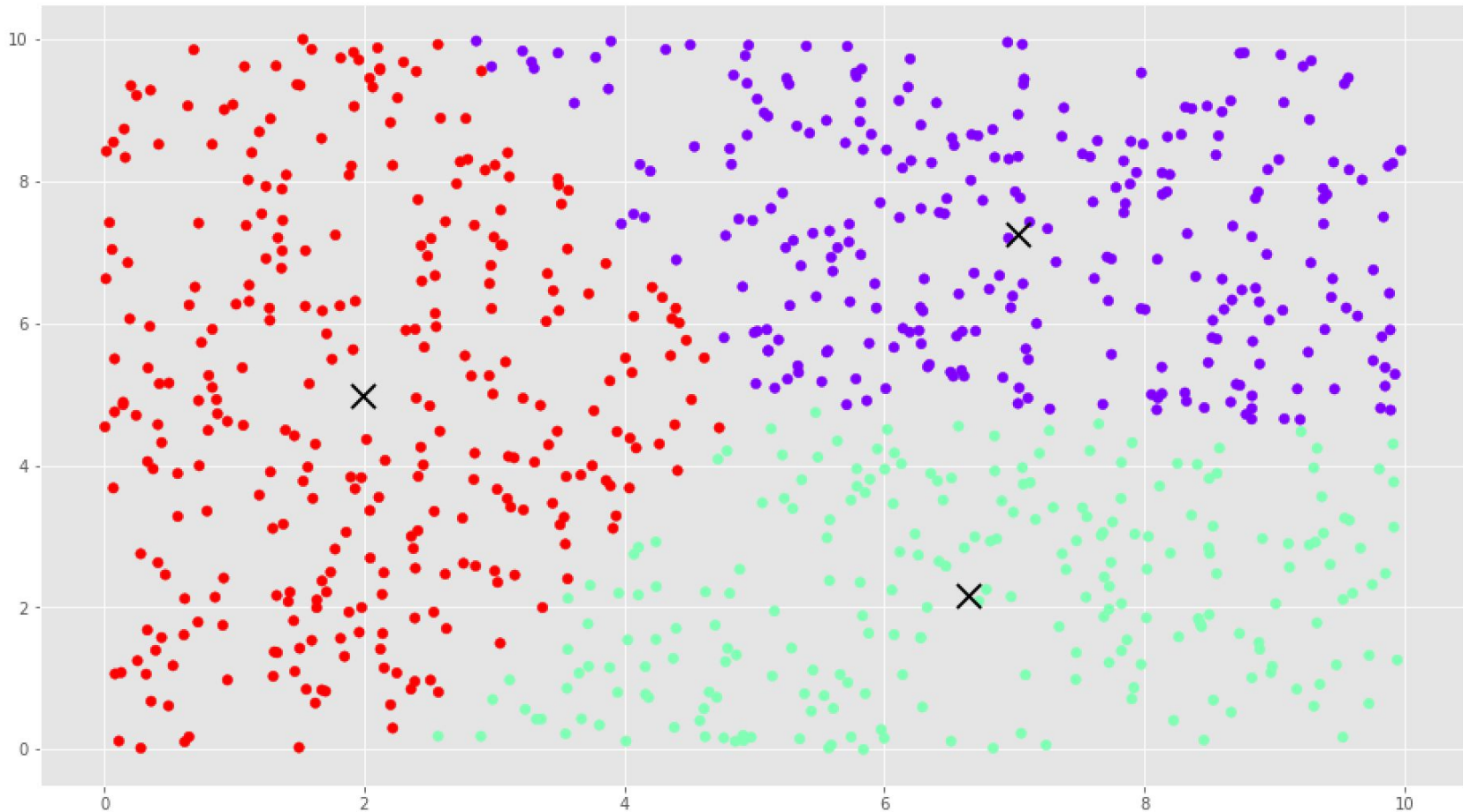
# Mostrar el resultado gráficamente
plt.scatter(valores_puntos[:, 0], valores_puntos[:, 1], c=valores_distancias, s=40, alpha=1, cmap=plt.cm.rainbow)
plt.plot(valores_centroides[:, 0], valores_centroides[:, 1], 'kx', markersize=15, mew=2)
plt.show()
```

Salida de la ejecución código anterior

```
In [21]: runfile('/Users/lfcastil 1 2/k means 3', wdir='/Users/lfcastil 1 2')
```

Centroides finales:

```
[[7.0319933 7.25119064]
 [6.64950566 2.17559501]
 [1.99578141 4.96810194]]
```



K-Means Clustering SKLEARN in Python

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(7)

x1 = np.random.standard_normal((100,2))*0.6+np.ones((100,2))
x2 = np.random.standard_normal((100,2))*0.5-np.ones((100,2))
x3 = np.random.standard_normal((100,2))*0.4-2*np.ones((100,2))+5
X = np.concatenate((x1,x2,x3),axis=0)

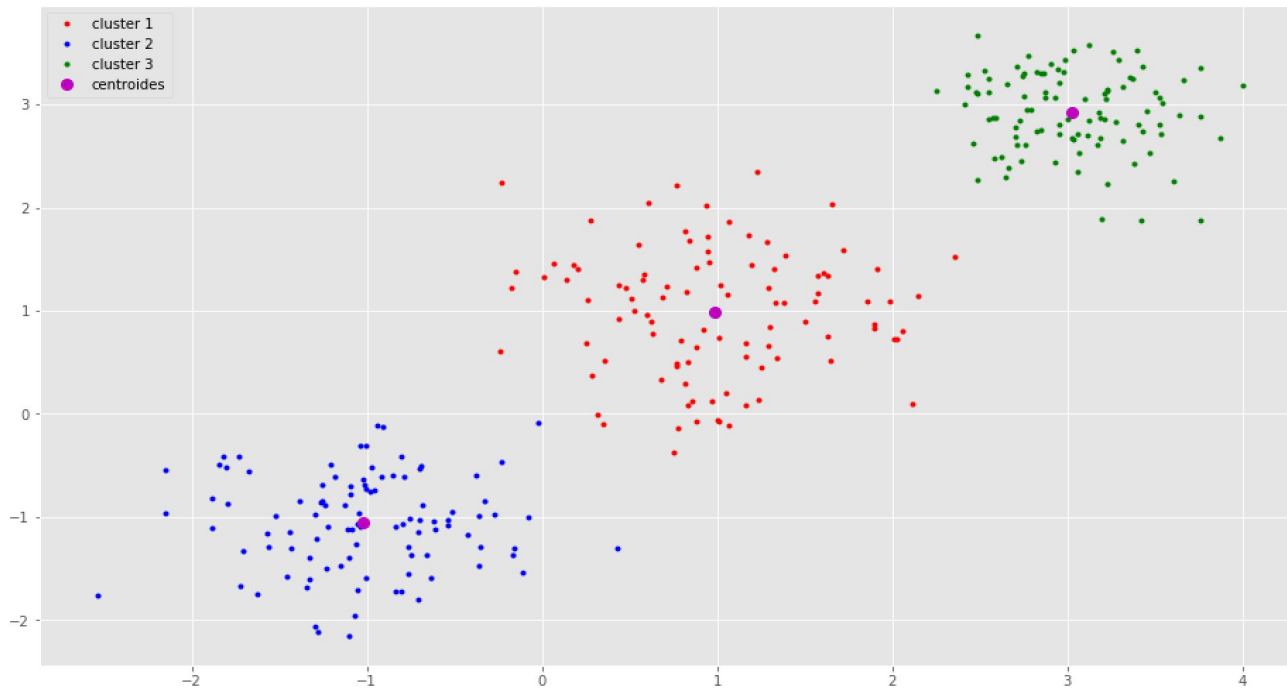
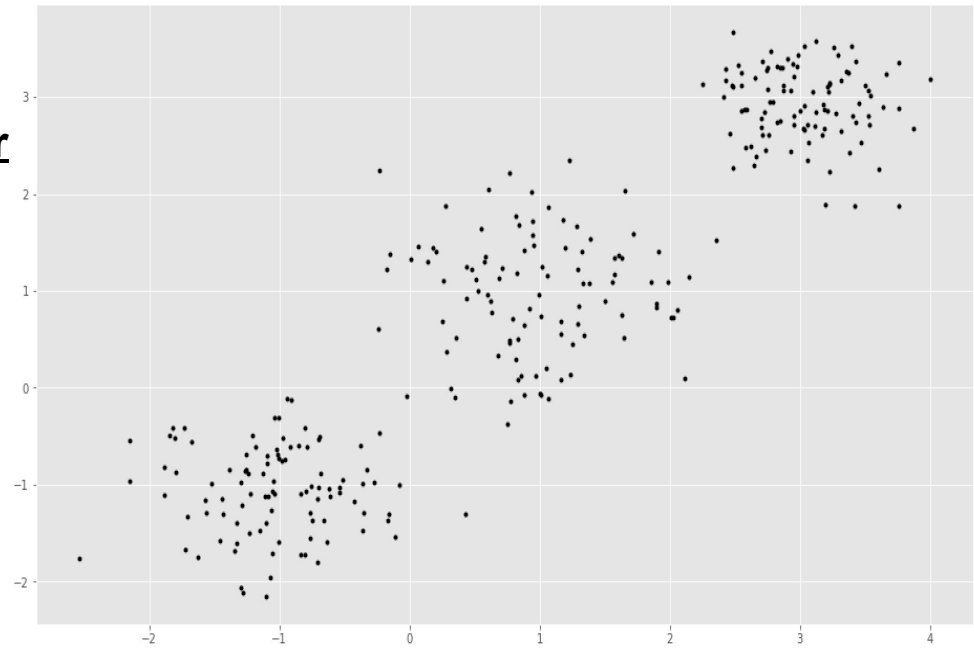
plt.plot(X[:,0],X[:,1],'k.')
plt.show()

from sklearn.cluster import KMeans
n = 3
k_means = KMeans(n_clusters=n)
k_means.fit(X)
centroides = k_means.cluster_centers_
etiquetas = k_means.labels_
plt.plot(X[etiquetas==0,0],X[etiquetas==0,1],'r.', label='cluster 1')
plt.plot(X[etiquetas==1,0],X[etiquetas==1,1],'b.', label='cluster 2')
plt.plot(X[etiquetas==2,0],X[etiquetas==2,1],'g.', label='cluster 3')

plt.plot(centroides[:,0],centroides[:,1],'mo',markersize=8, label='centroides')

plt.legend(loc='best')
plt.show()
```

Salida de la ejecución código anterior



Weaknesses of K-Mean Clustering

1. When the numbers of data are not so many, initial grouping will determine the cluster significantly.
2. The number of cluster, K , must be determined before hand. Its disadvantage is that it does not yield the same result with each run, since the resulting clusters depend on the initial random assignments.
3. We never know the real cluster, using the same data, because if it is inputted in a different order it may produce different cluster if the number of data is few.
4. It is sensitive to initial condition. Different initial condition may produce different result of cluster. The algorithm may be trapped in the local optimum.

Applications of K-Mean Clustering

- It is relatively *efficient and fast*. It computes result at **$O(tkn)$** , where n is number of objects or points, k is number of clusters and t is number of iterations.
- k-means clustering can be applied to *machine learning or data mining*
- Used on acoustic data in speech understanding to convert waveforms into one of k categories (known as Vector Quantization or Image Segmentation).
- Also used for choosing color palettes on old fashioned graphical display devices and Image Quantization.

CONCLUSION

- *K-means algorithm* is useful for undirected knowledge discovery and is relatively simple. K-means has found wide spread usage in lot of fields, ranging from unsupervised learning of neural network, Pattern recognitions, Classification analysis, Artificial intelligence, image processing, machine vision, and many others.
- Tomado de:
- <https://kjambi.kau.edu.sa>

References

- [Tutorial](#) - Tutorial with introduction of Clustering Algorithms (k-means, fuzzy-c-means, hierarchical, mixture of gaussians) + some interactive demos (java applets).
- Digital Image Processing and Analysis-byB.Chanda and D.Dutta Majumdar.
- H. Zha, C. Ding, M. Gu, X. He and H.D. Simon. "Spectral Relaxation for K-means Clustering", Neural Information Processing Systems vol.14 (NIPS 2001). pp. 1057-1064, Vancouver, Canada. Dec. 2001.
- J. A. Hartigan (1975) "Clustering Algorithms". Wiley.
- J. A. Hartigan and M. A. Wong (1979) "A K-Means Clustering Algorithm", Applied Statistics, Vol. 28, No. 1, p100-108.
- [D. Arthur](#), [S. Vassilvitskii](#) (2006): "How Slow is the k-means Method?,"
- D. Arthur, S. Vassilvitskii: "[k-means++ The Advantages of Careful Seeding](#)" 2007 Symposium on Discrete Algorithms (SODA).
- www.wikipedia.com

Dudas ?



@UdeCaldas

Luis F. Castillo Ossa @luisfercastillo